Verifiable Layered Architecture Visualizations

Invariants for Correct Level Computation

[Draft for co-author review]

Johannes Weigend
Weigend AM GmbH & Co.KG, Germany/Brazil
johannes.weigend@weigend.de

Veronika Schwarz
[Affiliation to be added]
veronika.schwarz@qaware.de

Michael Philippsen
Friedrich-Alexander-Universität Erlangen-Nürnberg, Germany
michael.philippsen@fau.de

Keywords: software architecture visualization, layout invariants, layered software city, strongly connected components, dependency analysis, software engineering tools

Abstract: Layered architecture visualizations communicate dependency structure through vertical position: elements that depend on others are placed higher. If the level-assignment pipeline is wrong, the picture can still look plausible while misleading the viewer: cyclic peer packages may be placed on different levels, or a regular dependency may point against the intended layer direction. Layout correctness should be the tool's responsibility, not the reader's—the implementation shows how hard that actually is.

Our 2021 IVAPP paper, *A Layered Software City for Dependency Visualization* (Dashuber et al., 2021), introduced a city layout in which the X/Y position of class buildings follows dependency-derived levels, while building height along the Z axis encodes orthogonal metrics such as complexity or lines of code. S202 reimplements and validates the dependency-level pipeline of that layout as an open-source Java tool under Apache 2.0, together with a Layout Invariant Checker that verifies the layout after every analysis run.

Correct level computation requires coordinating three interdependent constraint systems: class ordering, package dependency ordering, and level equality within cyclic groups. S202 separates two semantically distinct level concepts computed on different dependency graphs. Class levels are the longest path in the SCC-collapsed class dependency DAG; they represent global architectural depth independent of package structure. Package levels are derived from a weighted inter-package dependency graph whose edges carry method-call counts as weights; they represent the position of each package in the module dependency order. The two coordinate systems are independent: both classes and packages are assigned levels starting at zero, and a package's level is not constrained by the class levels of its contents. A horizontal row-ordering pass improves the legibility of dependency overlays without modifying either coordinate. Highly cyclic code bases are handled by a heuristic SCC breaker at the class level; its cut edges are not hidden but marked as violations.

The Layout Invariant Checker verifies four postconditions: non-heuristic class dependencies must point downward (R1); packages forming a cyclic group in the filtered package dependency graph must share a level (R2); in the weighted package graph, the dominant dependency direction must be reflected in the level order (R3); and edge classifications must match the current level and SCC state (R5). The checker distinguishes implementation bugs from tolerated heuristic violations and emits a copy-paste-ready reproducer block for every finding.

We will demonstrate the checker on real code bases, tracing the path from a marked layout element to a failing reproducer test. In the 92-module *software-ekg-7* system, an R2 finding exposed a missing package-SCC equalization step and directly triggered a pipeline fix. All four invariants were ported to an independent Unity/.NET 3D software city and produced identical findings on shared inputs. To our knowledge, no previous layered architecture visualization has provided this degree of machine-checkable self-validation; we also discuss generalization to other layered visualization techniques.

REFERENCES

Dashuber, V., Philippsen, M., and Weigend, J. (2021). A layered software city for dependency visualization. In *Proceedings of the 12th International Conference on Information Visualization Theory and Applications (IVAPP)*, VISIGRAPP 2021, pages 15–26. SCITEPRESS. Best Paper Award.

Eades, P., Lin, X., and Smyth, W. F. (1993). A fast and effective heuristic for the feedback arc set problem. *Information Processing Letters*, 47(6):319–323.

Karp, R. M. (1972). Reducibility among combinatorial problems. In Miller, R. E. and Thatcher, J. W., editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press.

Appendix A – Why Naive Single-Pass Does Not Work: Failure Modes

The Core Problem: Three Interacting Constraint Systems

A correct layered layout must satisfy three constraints simultaneously:

1. **Class constraint (R1):** If $A \to B$ denotes a dependency of A on B, then $\text{level}(A) > \text{level}(B)$.
2. **Package-order constraint (R3):** In the weighted inter-package dependency graph, the package with the dominant call volume must be placed at a strictly higher level than the package it calls into.
3. **SCC constraint (R2):** All classes in a cyclic dependency group receive the same level; packages that form a cyclic group in the filtered package dependency graph must also share a level.

Each constraint is individually manageable. Together they create a circular dependency system between the levels of computation themselves, not merely between classes.

Failure Mode 1: Cascading Level Updates

An algorithm iterates over classes and assigns levels. It first sees class A in package P_1, computes $\text{level}(A) = 3$, and therefore sets $\text{level}(P_1) = 3$. Later it reaches class B in package P_2 and discovers that A and B are in the same SCC and must share a level, now $\text{level}(A) = \text{level}(B) = 5$. This forces P_1 to be recomputed. If P_1 depends on P_3, P_3 may need to move as well. A cascade follows.

In a naive single-pass implementation, however, P_1 and P_3 may already have been finalized. SCC membership is only known after the relevant graph has been traversed; if level assignment and SCC discovery are interleaved, the iteration order decides whether the correction is still applied in time.

Failure Mode 2: Interleaved Cycle Detection Produces Classpath-Dependent Results

The most subtle practical failure mode occurs when cycle detection and level computation are not separated. The algorithm traverses the class graph, assigns levels, and handles discovered cycles on the fly, depending on which class happens to be processed first.

Classes come from JARs. The order in which the JVM sees classes follows the order in which they were packaged into the JAR. That order can depend on the build tool, module order in a multi-module build, filesystem ordering on the build server, and other implementation details. None of these orders is semantically meaningful; none encodes architectural dependency direction.

The result is unreliable: the same source code can produce different level layouts under different build configurations. The errors are subtle. The layout may still look plausible, arrows may mostly point in the expected direction, and no obvious visual break appears. Yet the precise hierarchy information is corrupted and may remain unnoticed for a long time.

The solution is to separate the phases strictly: first compute the SCC partition on the complete graph, then build the SCC-DAG, then assign levels. Tarjan's algorithm computes the SCC partition in $O(V+E)$; with deterministic input ordering, the implementation becomes reproducible and independent of classpath traversal artifacts.

Formal Counterexample: Why a Naive Recursive Algorithm Fails

The intuitive level-computation algorithm is a recursive DFS with memoization:

```
level(A):
  if A already computed: return level(A)
  if A has no dependencies: return 0
  return max(level(dep) for dep in A.dependencies) + 1
```

For acyclic graphs this is correct and equivalent to longest-path computation on a DAG. For cyclic graphs it fails structurally.

Counterexample with two SCCs:

```
SCC_1 = {A, B}, with A -> B -> A
SCC_2 = {C, D}, with C -> D -> C
Cross dependency: A -> C
```

The correct levels are $\text{level}(C) = \text{level}(D) = 0$ and $\text{level}(A) = \text{level}(B) = 1$.

Recursive algorithm, starting at A:

```
level(A)  [A "in progress", sentinel = 0]
  -> level(B)
      -> level(A): cycle detected, return 0
    B.level = 0 + 1 = 1
  -> level(C)  [C "in progress", sentinel = 0]
      -> level(D)
          -> level(C): cycle detected, return 0
        D.level = 0 + 1 = 1
    C.level = 1 + 1 = 2   <-- wrong; correct would be 0
  A.level = max(1, 2) + 1 = 3   <-- wrong; correct would be 1
```

After SCC equalization, $\text{SCC}_1 = \max(3,1) = 3$ and $\text{SCC}_2 = \max(2,1) = 2$. Both results are wrong.

The cause is that the recursive algorithm enters SCC_2 from the context of SCC_1. The sentinel-based local cycle handling inflates the level of SCC_2, and that inflated level then feeds into the computation of SCC_1. The correct level of SCC_2, namely zero, is only known after the graph has been collapsed into SCCs and evaluated as an SCC-DAG.

Thus, recursive level algorithms that mix traversal, cycle handling, and level assignment can produce traversal-order-dependent results on graphs with interacting SCCs. In JVM-based tools, that traversal order can be influenced by classpath and packaging order. The correct solution collapses SCCs before level assignment: Tarjan, then SCC-DAG, then longest path.

Failure Mode 3: Deriving Package Levels from Class Levels

An obvious implementation assigns each package a level equal to the maximum level of its contained classes, then lifts parent packages transitively. This is wrong for two reasons.

First, the rule conflates containment with dependency. A package that *contains* a class at level 7 is not necessarily architecturally positioned at level 7 itself; it may have no cross-package dependencies at all. Two disconnected package trees processed from the same JAR would be ordered relative to each other by accident of their class contents, not by their actual dependency relationships.

Second, the approach creates a circular dependency between the two level systems: class levels depend on the class graph, but package levels derived from class levels then constrain the visual position of packages relative to classes, making the combined coordinate system ill-defined.

S202 avoids this by computing class levels and package levels on independent graphs with no mutual constraint. Each system starts at zero and grows according to its own dependency structure.

Failure Mode 4: The Big Lake – SCCs Without Heuristics Collapse the Layout

Without a mechanism for breaking large SCCs, all members of a cyclic group must be placed on the same level. For small cycles of two to five classes this is acceptable. For Minecraft Forge, however, we observed a single SCC containing 4,038 classes. Without a breaker, more than 40 percent of the project would land on one flat level at the bottom of the layout. Everything else would stand above it, but the interesting internal structure of the core would be invisible.

The heuristic SCC breaker solves this practical problem. It identifies back edges, meaning dependencies that run against the natural architectural flow, and temporarily removes them for level computation. The removed edges are visualized as violations instead of being ignored. The layout remains honest about architectural debt while still producing a usable hierarchy.

Relation to Feedback Arc Set Heuristics

Failure Mode 4 is an instance of the Minimum Feedback Arc Set problem: remove a minimal set of directed edges so the graph becomes acyclic. The problem is NP-hard (Karp, 1972), so any practical polynomial-time implementation must use a heuristic or approximation.

A well-known related heuristic is due to Eades, Lin, and Smyth (Eades et al., 1993). It orders nodes according to the difference between outgoing and incoming degree, then classifies edges that point backward in the resulting sequence as feedback edges. The intuition is similar to S202: high out-degree suggests a higher-level element; high in-degree suggests a foundational element.

S202 uses a deliberately simpler variant: a direct normalized score per node,

$$r(P) = \frac{\text{outWeight}(P) - \text{inWeight}(P)}{\max(1,\ \text{outWeight}(P) + \text{inWeight}(P))},$$

where the weights are summed within the SCC under consideration. At the class level the weights are the in- and out-degrees in the class dependency graph; at the package level they are aggregated method-call counts. An edge $P_A \to P_B$ inside an SCC is classified as a back-edge candidate when $r(P_A) < r(P_B) - 0.1$. Equal-rank pairs represent symmetric bidirectional dependencies with no dominant architectural direction; they remain in the same SCC and are assigned the same level.

The important distinction is transparency. Many tools break cycles implicitly, but S202 classifies heuristic cut edges explicitly. They are marked as `VIOLATION`, rendered in red, and known to the invariant checker. This makes algorithmic defects separable from tolerated heuristic artifacts.

The Solution: Two Semantically Distinct Level Concepts on Independent Graphs

Failure modes 1 to 3 share a root cause: conflating level concepts that belong to different semantic domains. S202 resolves this by computing two independent level coordinate systems.

- **Class levels** are the longest path in the SCC-collapsed class dependency DAG. They represent global architectural depth: how many layers of dependencies separate a class from the foundation of the analyzed system.
- **Package levels** are the longest path in the SCC-collapsed weighted inter-package dependency graph. They represent the position of each package module relative to the packages it calls into, independently of the class-level coordinate.

Both concepts use the same algorithmic shape—Tarjan SCC detection, back-edge identification, DAG condensation, longest-path propagation—but on different graphs with different edge weights. Because the two coordinate systems are independent, packages with no cross-package dependencies correctly land at level 0 regardless of the class levels inside them. Two disconnected package trees processed from the same JAR are each levelled from zero in their own connected component; no false ordering between them is induced.

The separation of class levels and package levels is not a design choice—no correct level-computation pipeline can derive both coordinate systems from a single graph in a single pass.

The two phases have the following algorithmic structure:

Algorithm 1: Phase 1 – Global Class Levels (`LevelCalculator`, Step 3)

class dependency graph
→ Tarjan (SCCs)
→ SCCBreaker: identify heuristic back-edges in large SCCs ($|S| \geq 3$)
rank$(P) = (\text{outDeg} - \text{inDeg}) \,/\, \max(1, \text{outDeg} + \text{inDeg})$
edge $A \to B$ is a back-edge candidate if rank$(A) <$ rank$(B) - 0.1$
→ Remove back-edges → filtered class DAG
→ Condense into SCC-DAG → longest-path pass
→ Assign SCC level to all member classes (equalization implicit)

Algorithm 2: Phase 2 – Package Levels (`LevelCalculator`, Step 4)

Build weighted inter-package dependency graph $G_P = (V_P, E_P, w)$:
For each class C in package P_{leaf} with method calls to class D in P_{target}
($P_{\text{leaf}} \neq P_{\text{target}}$, and P_{target} not a descendant of P_{leaf}):
$w(P_{\text{ancestor}}, P_{\text{target}})$ += callCount$(C \to D)$
for every ancestor P_{ancestor} of P_{leaf} that is not a subtree ancestor of P_{target}
Fallback weight = 1 for structural references with no method-call data

Iterative SCC-breaking (repeat until stable):
Tarjan on current graph → find SCCs with $|S| \geq 2$
For each member $P \in S$:
$r(P) = \left(\sum_{Q \in S} w(P,Q) - \sum_{Q \in S} w(Q,P)\right) \,/\, \max\left(1, \sum_{Q \in S} w(P,Q) + \sum_{Q \in S} w(Q,P)\right)$
Edge $P_A \to P_B$ inside S is a back-edge if $r(P_A) < r(P_B) - 0.1$; remove it
Equal-rank pairs remain in the SCC → assigned the same level

Level assignment:
Condense remaining graph into SCC-DAG
Child–parent lift: for each package $P_c \subset P_p$ (containment),
effective dep height of $P_p \leftarrow \max\left(P_p.\text{pkgLevel}, \max\{l_{\text{class}}(D) \mid C \in P_c,\ C \xrightarrow{\text{non-back}} D \in P_p\}\right)$
Longest-path propagation on SCC-DAG
Assign SCC level to all member packages

Phase 1 is reproducible because SCCs are collapsed before levels are assigned. Phase 2 treats class levels and package levels as independent coordinate systems: a package with no outgoing cross-package calls lands at level 0, even if its contained classes are at high class levels. The child–parent lift in Phase 2 is the only point where class levels influence package levels, and even then only for the specific classes that a child package calls in its parent—not for all classes in the parent package. A separate horizontal row-ordering pass (Phase 3, described in the Implementation section) improves legibility of dependency overlays without modifying either coordinate.

	Phase 1 – Class Levels	**Phase 2 – Package Levels**
Graph	Class dependency graph	Weighted inter-package graph
Edge weight	Unweighted (structural dep)	Aggregated method-call counts
Meaning	Architectural depth in class DAG	Module dependency position
Zero point	Leaf classes (no dependencies) = 0	Packages with no outgoing calls = 0
Cycles	SCCBreaker (degree heuristic)	Weight-based rank heuristic
Independence	Does not use package levels	Does not derive from class levels

Table 1: The two level concepts computed by S202.

Layout Invariants for Level Correctness

The two level phases define four machine-checkable postconditions verified by the Layout Invariant Checker after every analysis run:

Rule	**Postcondition**
R1	Non-heuristic class dependencies point strictly downward: $A \to B \Rightarrow \text{level}(A) > \text{level}(B)$
R2	All packages forming a cyclic group in the filtered package dependency graph share the same level
R3	For every weighted package edge (P_A, P_B) with $w(P_A,P_B) > w(P_B,P_A)$ that is not a package back-edge: $\text{level}(P_A) > \text{level}(P_B)$
R5	Edge classifications (`NORMAL`, `VIOLATION`, `BACK_EDGE`) are consistent with the current level assignment and SCC membership

Table 2: The four layout invariants verified after every analysis run.

When we deployed the invariant checker against large real-world code bases, it guided refinement of both the pipeline and the rules themselves. Against Minecraft Forge (2 895 classes, 220 packages), the checker initially

reported two R2 violations. Investigation showed these were *false positives*: R2 filtered class-level heuristic back-edges but not the package-level back-edges that the package SCC-breaker had already removed. R3 already applied the correct filter; once R2 was aligned, the Minecraft findings disappeared. Figure 1 shows the original R2 output before this refinement.

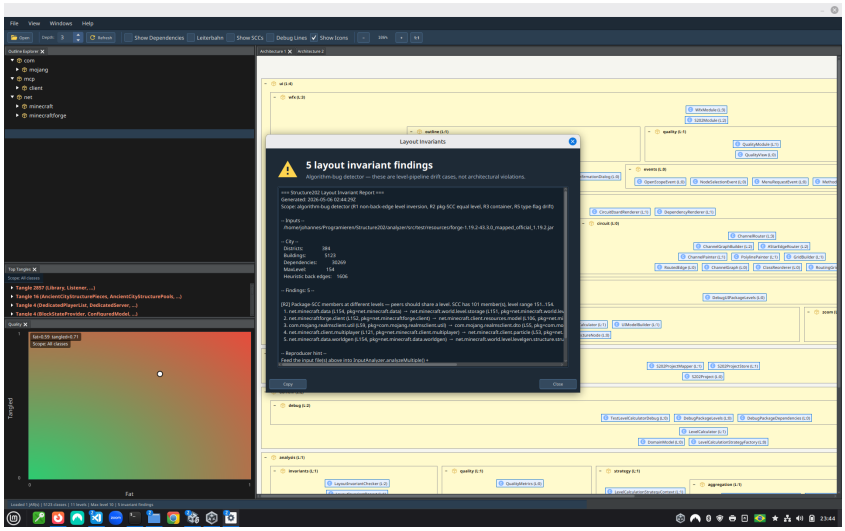


Figure 1: R2 checker output on Minecraft Forge before the package back-edge filter was added. The report was a false positive: the packages were correctly broken out of their SCC by the package-level heuristic—R2 had not yet been taught to respect that cut. False positives of this kind forced the rule to become more precise.

Against *software-ekg-7*, in contrast, R2 surfaced a genuine pipeline bug: a missing package-SCC equalization step had left peer packages at different levels. This finding was only detectable through automated structural analysis; the checker's own 155-test suite was fully green throughout. The invariant checker connects the visual symptom to a reproducible algorithm defect.

The Remaining Rules: R1, R3, and R5

R1 ($A \to B \Rightarrow \text{level}(A) > \text{level}(B)$) is the most direct invariant: it is a corollary of longest-path computation on the SCC-DAG and holds trivially for any correctly implemented class-level assignment. Its value lies in providing an immediate sanity check—a failing R1 signals a fundamental breakdown in the class-level pipeline.

R3 verifies that the dominant direction in the weighted package dependency graph is reflected in the level assignment. For any package edge (P_A, P_B) where $w(P_A, P_B) > w(P_B, P_A)$—meaning P_A calls into P_B significantly more than vice versa—the invariant requires $\text{level}(P_A) > \text{level}(P_B)$. Package back-edges identified during SCC-breaking are excluded from this check, analogously to how R1 excludes class-level heuristic back-edges. A failing R3 indicates that the weight-based SCC-breaking cut the wrong package edge, or that the package-level DAG propagation did not converge correctly.

R5 validates that every edge's stored classification (`NORMAL`, `VIOLATION`, `BACK_EDGE`) is still consistent with the current level assignment and SCC membership. Edge labels are computed during analysis and cached in the model; after any algorithmic change they can silently become stale. R5 is therefore a cross-cutting consistency guard: it ties the structural output of the level pipeline back to the semantic annotations visible to the user.

R4: A Retired Rule and What It Teaches

An early version of the invariant checker contained R4:

```
Cyclic child packages must share a level.
```

The rule sounds reasonable and is correct in simple cases. In practice it failed because the raw package graph contains relationships that should not be interpreted as peer-level architectural cycles:

- **Parent-child edges** create apparent bidirectional relationships because a parent contains its children and children belong to their parent. Without filtering, many parent/child pairs look cyclic.
- **SCCBreaker back edges** connect packages that are not necessarily architectural peers. A rule that includes these edges equalizes the wrong candidates.
- **Shared class SCCs** can make two packages appear package-dependent without representing a true structural peer relationship.

R4 was replaced by R2, which expresses the same intention on the correctly filtered graph. R2 removes class-level heuristic back-edges, package-level SCC-breaking back-edges, intra-subtree edges, and edges caused by shared class SCCs before running Tarjan on the remaining package graph.

Specifying invariants for a three-level constraint system turns out to be harder than it appears. The checker therefore provides a second safety net: it not only tests the implementation against the rules, but also exposes when the rules themselves are too broad or too coarse. False positives force the rule to become more precise. The later R2 finding in *software-ekg-7* shows that the refined rule was precise enough to distinguish real pipeline bugs from apparent ones.

Implementation: The Complete Pipeline

The conceptual structure expands into the following concrete implementation steps, which constitute the reference implementation (Appendix B).

Algorithm 1: Complete Level Pipeline (`LevelCalculator`)

Step 1: Create class objects (all levels $\leftarrow 0$)
Step 2: Create package objects (all levels $\leftarrow 0$)

Step 3: Compute class levels (Phase 1)
$\rightarrow$ Tarjan on class dependency graph
$\rightarrow$ SCCBreaker: identify heuristic back-edges in large SCCs ($|S| \geq 3$):
$r(C) = (\text{outDeg} - \text{inDeg}) \,/\, \max(1, \text{outDeg} + \text{inDeg})$;
edge $A \rightarrow B$ is a back-edge candidate if $r(A) < r(B) - 0.1$
$\rightarrow$ Remove identified back-edges $\rightarrow$ filtered class DAG
$\rightarrow$ Condense into SCC super-nodes
$\rightarrow$ Single longest-path pass on condensed DAG
$\rightarrow$ Assign SCC level to all member classes

Step 4: Compute package levels (Phase 2)
4a. Build weighted inter-package dependency graph:
$w(P_A, P_B) = \sum$ method-call counts from subtree(P_A) to P_B;
propagate weights up to all ancestor packages;
child$\rightarrow$ancestor and sibling edges included; parent$\rightarrow$child excluded

4b. Iterative package SCC-breaking (repeat until stable):
Tarjan on current graph $\rightarrow$ SCCs with $|S| \geq 2$;
$r(P) = (\sum_{Q \in S} w(P,Q) - \sum_{Q \in S} w(Q,P)) \,/\, \max(1, \ldots)$;
edge $P_A \rightarrow P_B$ is a back-edge if $r(P_A) < r(P_B) - 0.1$; remove it;
equal-rank pairs remain in the same SCC $\rightarrow$ assigned the same level

4c. Level assignment on condensed package SCC-DAG:
child–parent lift: effective dep height of P_p from child P_c is

$$\max\big(P_p.\text{pkgLevel},\ \max\{l_{\text{class}}(D) \mid C \in P_c,\ C \xrightarrow{\text{non-back}} D \in P_p\}\big);$$

longest-path propagation; assign SCC level to all member packages

Step 5: Set reverse dependencies (dependents)

After Step 3, a class's level is the level of the SCC node to which it belongs, measured as the longest path in the cycle-free quotient graph across all classes of the analyzed project. Cycles do not influence level assignment internally because they have been collapsed before longest-path computation begins.

Step 4 treats class levels and package levels as independent coordinate systems. A package with no outgoing cross-package calls receives level 0, regardless of the class levels inside it. Two package trees with no shared dependencies—even if analyzed from the same JAR—are each levelled from zero in their own connected component of the weighted package graph. The child–parent lift in Step 4c is the only place where class-level information influences package levels, and only for the specific classes that a child package directly calls in its parent.

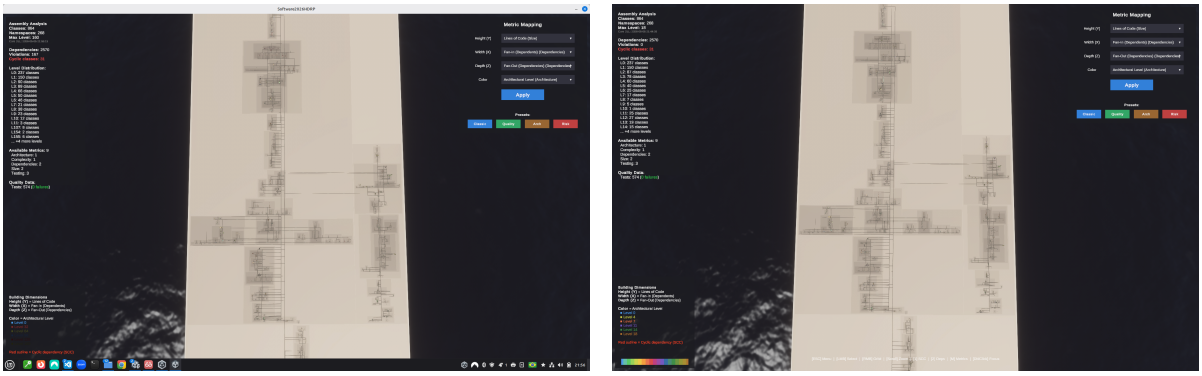
Algorithm 2: Phase 3 – Horizontal Row Ordering (`HorizontalRowLayoutOptimizer`)

for each package *P*:
  Step 1: Group direct children by their level
  Step 2: Collect subtree classes per row element
  Step 3: Build weighted undirected proximity links from inter-sibling class dependencies
  Step 4: Run bounded barycentric sweeps across rows
  Step 5: Store the result in `horizontalLayoutOrder`
rendering: sort a copied child view by level, then `horizontalLayoutOrder`, then full name

Phase 3 does not change any level assignment and is deliberately excluded from the layout invariants. Its only purpose is to improve legibility when dependency overlays are displayed: same-level elements with dependencies to adjacent levels are placed horizontally near each other. The implementation stores this as a separate `horizontalLayoutOrder` field and never mutates the semantic child list.

R2 Fix in *software-ekg-7*: Visual Evidence

The *software-ekg-7* case is the concrete failure that motivated the package-level pipeline. Before the fix, the city view still looked plausible: the package rectangles and dependency streets formed a readable layout, and no single visual element made the pipeline defect obvious. The metrics panel, however, exposed the inconsistency: 167 dependency violations were reported and the maximum architectural level had inflated to 160. After the package-level equalization was corrected, the same input produced zero violations and a maximum level of 18. The number of classes, packages, dependencies, cyclic classes, and tests stayed constant; the change is therefore attributable to level propagation, not to a different input model.



(a) Before the fix: the full city view looks plausible, but the metric panel reports 167 violations and MaxLevel 160.

(b) After the fix: the same analyzed system reports zero violations and MaxLevel 18.

Figure 2: Full-screen before/after screenshots of the *software-ekg-7* software city. The visible city changes only subtly; the level metrics reveal the correction.

(a) Before: 167 violations, MaxLevel 160.

(b) After: zero violations, MaxLevel 18.

Figure 3: Cropped metric panels from Figure 2. These panels are the clearest user-visible symptom of the propagation defect and its fix.

Summary

Correct level computation for layered architecture visualizations requires satisfying three interacting constraint systems simultaneously: class dependency ordering (R1), package dependency ordering (R3), and SCC level equality (R2). No single traversal pass can satisfy all three; the failure modes documented above are structural, not accidental.

S202 resolves this by computing two semantically distinct level coordinates on independent dependency graphs. Class levels are the longest path in the SCC-collapsed class dependency DAG; they represent global architectural depth. Package levels are derived from a weighted inter-package dependency graph whose edges carry aggregated method-call counts; they represent the module dependency position independently of class levels. Both coordinate systems use the same algorithmic shape—Tarjan SCC detection, weight-based back-edge identification, DAG condensation, longest-path propagation—on their respective graphs.

A horizontal row-ordering pass (Phase 3) improves the legibility of dependency overlays without modifying either coordinate or affecting any layout invariant. The Layout Invariant Checker verifies four machine-checkable postconditions (R1, R2, R3, R5) after every analysis run and emits copy-paste-ready reproducer blocks for each finding.

The *software-ekg-7* case demonstrates that the checker catches defects invisible to visual inspection: a plausible city layout reported 167 violations and a maximum architectural level of 160; after the package-level equalization was corrected, the same input produced zero violations and a maximum level of 18, with no change to the analyzed source code.

Appendix B – LevelCalculator Reference Implementation

The following source excerpt is the complete Java implementation of the level pipeline discussed above.

```
package de.weigend.s202.domain;

import de.weigend.s202.analysis.scc.SCCBreaker;
import de.weigend.s202.analysis.scc.StronglyConnectedComponent;
import de.weigend.s202.analysis.scc.TarjanSCCFinder;
import de.weigend.s202.analysis.strategy.LevelCalculationStrategyContext;
import de.weigend.s202.analysis.strategy.impl.HeuristicSCCBreakingStrategy;
import de.weigend.s202.reader.DependencyModel;

import java.util.*;
import java.util.logging.Logger;

/**
 * Calculates architectural levels for classes and packages based on dependencies.
 *
 * Pipeline:
 *   Step 1  Create class objects       (all levels = 0)
 *   Step 2  Create package objects     (all levels = 0)
 *   Step 3  Compute class levels       strategy (Tarjan → SCC-break → DAG → longest-path)
 *   Step 4  Compute package levels     weighted inter-package graph → SCC-break → DAG → longest-path
 *                                      + child→parent lift for class-level alignment
 *   Step 5  Set reverse dependencies
 *
 * Package levels are computed independently from class levels using a weighted
 * inter-package dependency graph. The weight of an edge P_A → P_B is the number
 * of distinct classes in P_A that depend on at least one class in P_B (intra-subtree
 * edges excluded). This separates the containment hierarchy from the dependency
 * hierarchy and correctly handles disconnected package trees (they are levelled
 * independently, both starting at 0).
 *
 * Package SCC-breaking uses a weight-based rank score:
 *   rank(P) = (Σ outgoing weights within SCC − Σ incoming weights within SCC)
 *             / max(1, sum of both)
 * Edge P_A→P_B is a back-edge when rank(P_A) < rank(P_B) − 0.1.
 * Equal-rank pairs (symmetric dependency) are left in the same SCC and assigned
 * the same level -- they are genuine peers with no dominant dependency direction.
 */
public class LevelCalculator {

    private static final Logger LOG = Logger.getLogger(LevelCalculator.class.getName());
    private static final double RANK_THRESHOLD = 0.1;

    private final LevelCalculationStrategyContext strategyContext;

    public LevelCalculator() {
        this(LevelCalculationStrategyFactory.createDefault());
    }

    public LevelCalculator(LevelCalculationStrategyContext strategyContext) {
        Objects.requireNonNull(strategyContext, "strategyContext cannot be null");
        this.strategyContext = strategyContext;
    }

    public DomainModel calculate(DependencyModel rawModel) {
        DomainModel model = new DomainModel();

        // Step 1: Create class objects (all levels = 0)
        for (String className : rawModel.getAllClassNames()) {
            DependencyModel.ClassInfo rawClass = rawModel.getClass(className);
```

```
            model.addClass(className, new DomainModel.CalculatedElementInfo(
                className, rawClass.simpleName, "CLASS", 0,
                new HashSet<>(rawClass.dependencies), rawClass.interfaceType));
        }

        // Step 2: Create package objects (all levels = 0)
        for (String packageName : rawModel.getAllPackageNames()) {
            DependencyModel.PackageInfo rawPkg = rawModel.getPackage(packageName);
            model.addPackage(packageName, new DomainModel.CalculatedElementInfo(
                packageName, rawPkg.simpleName, "PACKAGE", 0, new HashSet<>()));
        }

        // Step 3: Compute class levels
        calculateClassLevels(model);

        // Step 4: Compute package levels from weighted inter-package graph.
        // Class back-edges are passed so child→parent lifting excludes them.
        Set<SCCBreaker.Edge> classBackEdges = classBackEdgesFromStrategy();
        calculatePackageLevels(model, rawModel, classBackEdges);

        // Step 5: Set reverse dependencies
        updateDependentRelationships(model);

        return model;
    }

    public LevelCalculationStrategyContext getStrategyContext() {
        return strategyContext;
    }

    private Set<SCCBreaker.Edge> classBackEdgesFromStrategy() {
        var strategy = strategyContext.getClassLevelStrategy();
        if (strategy instanceof HeuristicSCCBreakingStrategy h) {
            return h.getLastIdentifiedBackEdges();
        }
        return Set.of();
    }

    // ------------------------------------------------------------------------
    // Step 3 -- class levels
    // ------------------------------------------------------------------------

    private void calculateClassLevels(DomainModel model) {
        Map<String, Set<String>> classDeps = new HashMap<>();
        for (DomainModel.CalculatedElementInfo c : model.getAllClasses().values()) {
            classDeps.put(c.fullName, new HashSet<>(c.dependencies));
        }
        Map<String, Integer> levels =
            strategyContext.getClassLevelStrategy().calculateClassLevels(classDeps);
        for (Map.Entry<String, Integer> e : levels.entrySet()) {
            DomainModel.CalculatedElementInfo info = model.getClass(e.getKey());
            if (info != null) info.setLevel(e.getValue());
        }
    }

    // ------------------------------------------------------------------------
    // Step 4 -- package levels via weighted inter-package graph
    // ------------------------------------------------------------------------

    private void calculatePackageLevels(DomainModel model, DependencyModel rawModel,
                                        Set<SCCBreaker.Edge> classBackEdges) {
        if (model.getAllPackages().isEmpty()) return;
```

```
        Set<String> allPkgNames = model.getAllPackages().keySet();

        // Build weighted graph: weight[from][to] = # distinct classes in 'from'
        // with at least one dependency on a class in 'to' (subtree edges excluded).
        Map<String, Map<String, Integer>> weights = buildWeightedPackageGraph(model, rawModel);

        // Unweighted adjacency for Tarjan (direction matters, zero-weight edges excluded)
        Map<String, Set<String>> graph = new LinkedHashMap<>();
        for (String pkg : allPkgNames) graph.put(pkg, new LinkedHashSet<>());
        for (Map.Entry<String, Map<String, Integer>> entry : weights.entrySet()) {
            graph.get(entry.getKey()).addAll(entry.getValue().keySet());
        }

        // Iteratively break SCCs using weight-based rank, then assign levels
        // via SCC-collapsed DAG longest-path. Remaining SCCs (equal-rank peers)
        // are collapsed to the same level without cutting any edge.
        Set<String> backEdgeKeys = new LinkedHashSet<>(); // "from\0to" -- tracked for R3
        boolean changed = true;
        while (changed) {
            changed = false;
            for (StronglyConnectedComponent scc : new TarjanSCCFinder(graph).findSCCs()) {
                if (scc.getSize() < 2) continue;
                Set<String> members = scc.getMembers();

                // Compute weight-based rank for each member within this SCC
                Map<String, Double> rank = new HashMap<>();
                for (String m : members) {
                    int out = 0, in = 0;
                    for (String other : members) {
                        if (other.equals(m)) continue;
                        out += weights.getOrDefault(m, Map.of()).getOrDefault(other, 0);
                        in  += weights.getOrDefault(other, Map.of()).getOrDefault(m, 0);
                    }
                    rank.put(m, (out - in) / (double) Math.max(1, out + in));
                }

                // Identify back-edges: from→to where rank(from) < rank(to) − threshold
                for (String from : new ArrayList<>(members)) {
                    for (String to : new ArrayList<>(graph.getOrDefault(from, Set.of()))) {
                        if (!members.contains(to)) continue;
                        if (rank.get(from) < rank.get(to) - RANK_THRESHOLD) {
                            String key = from + "\0" + to;
                            if (backEdgeKeys.add(key)) {
                                graph.get(from).remove(to);
                                changed = true;
                            }
                        }
                    }
                }
            }
        }

        // Assign package levels: SCC-collapsed DAG → longest-path
        List<StronglyConnectedComponent> sccs = new TarjanSCCFinder(graph).findSCCs();
        Map<String, StronglyConnectedComponent> pkgToScc = new HashMap<>();
        for (StronglyConnectedComponent scc : sccs) {
            for (String m : scc.getMembers()) pkgToScc.put(m, scc);
        }

        // SCC dependency graph (between SCC IDs)
        Map<Integer, Set<Integer>> sccDeps = new HashMap<>();
        for (StronglyConnectedComponent scc : sccs) {
```

```
            sccDeps.put(scc.getId(), new HashSet<>());
            for (String m : scc.getMembers()) {
                for (String to : graph.getOrDefault(m, Set.of())) {
                    StronglyConnectedComponent toScc = pkgToScc.get(to);
                    if (toScc != null && toScc.getId() != scc.getId()) {
                        sccDeps.get(scc.getId()).add(toScc.getId());
                    }
                }
            }
        }

        // Child→parent lift: for each package P, find the max class-level of classes
        // in P that are depended on by classes in CHILD packages of P (sub-packages).
        // SCC back-edges are excluded. This ensures figurapi.primitive ends up strictly
        // above figurapi.Rotation (class L1), while external deps are unaffected.
        Map<String, Integer> childPkgUsedLevel = new HashMap<>();
        for (DomainModel.CalculatedElementInfo cls : model.getAllClasses().values()) {
            String fromPkg = extractPackageName(cls.fullName);
            if (fromPkg == null) continue;
            for (String dep : cls.dependencies) {
                String toPkg = extractPackageName(dep);
                if (toPkg == null || fromPkg.equals(toPkg)) continue;
                // Only child→parent edges: fromPkg is a sub-package of toPkg
                if (!fromPkg.startsWith(toPkg + ".")) continue;
                // Skip class-level back-edges
                if (classBackEdges.contains(new SCCBreaker.Edge(cls.fullName, dep))) continue;
                DomainModel.CalculatedElementInfo depCls = model.getAllClasses().get(dep);
                if (depCls == null) continue;
                childPkgUsedLevel.merge(toPkg, depCls.level, Math::max);
            }
        }

        // Longest-path levels on SCC-DAG.
        // For child→parent package edges, effective dep height =
        //   max(pkgLevel, childPkgUsedLevel) so the dependent lands above the classes
        //   it uses from its parent package. External cross-package deps use pkgLevel only.
        Map<Integer, Integer> sccLevels = new HashMap<>();
        for (StronglyConnectedComponent scc : sccs) sccLevels.put(scc.getId(), 0);
        boolean lvlChanged = true;
        while (lvlChanged) {
            lvlChanged = false;
            for (StronglyConnectedComponent scc : sccs) {
                int maxDep = -1;
                for (int depId : sccDeps.getOrDefault(scc.getId(), Set.of())) {
                    int depPkgLevel = sccLevels.getOrDefault(depId, 0);
                    int effectiveDep = depPkgLevel;
                    // Check if any member of this SCC is a child of any member of depSCC
                    for (String fromMember : scc.getMembers()) {
                        for (Map.Entry<String, StronglyConnectedComponent> e : pkgToScc.entrySet()) {
                            if (e.getValue().getId() != depId) continue;
                            String depMember = e.getKey();
                            if (fromMember.startsWith(depMember + ".")) {
                                effectiveDep = Math.max(effectiveDep,
                                    childPkgUsedLevel.getOrDefault(depMember, 0));
                            }
                        }
                    }
                    maxDep = Math.max(maxDep, effectiveDep);
                }
                int newLevel = maxDep >= 0 ? maxDep + 1 : 0;
                if (sccLevels.get(scc.getId()) != newLevel) {
                    sccLevels.put(scc.getId(), newLevel);
```

```
                    lvlChanged = true;
                }
            }
        }

        // Apply levels to all packages
        Map<String, DomainModel.CalculatedElementInfo> pkgInfos = model.getAllPackages();
        for (StronglyConnectedComponent scc : sccs) {
            int level = sccLevels.get(scc.getId());
            for (String member : scc.getMembers()) {
                DomainModel.CalculatedElementInfo pkg = pkgInfos.get(member);
                if (pkg != null) pkg.setLevel(level);
            }
        }

        // Store the weighted graph and identified back-edges in the model
        model.setPackageEdgeWeights(weights);
        model.setPackageBackEdges(backEdgeKeys);

        // Populate package dependencies (unweighted) for reverse-dependency tracking
        for (Map.Entry<String, Map<String, Integer>> entry : weights.entrySet()) {
            DomainModel.CalculatedElementInfo pkg = pkgInfos.get(entry.getKey());
            if (pkg != null) entry.getValue().keySet().forEach(pkg::addDependency);
        }
    }

    /**
     * Builds the weighted inter-package dependency graph.
     * weight(P_A → P_B) = total method-call count from classes in P_A's subtree
     * to classes in P_B. Method calls are a stronger signal than distinct-class
     * counts: a class called hundreds of times clearly dominates one called once,
     * making SCC-breaking direction unambiguous. Intra-subtree calls are excluded.
     * Child-to-ancestor edges (e.g. sub-package calling into its parent package)
     * are included; parent-to-child edges are excluded.
     * Each call count is propagated to all ancestor packages up to the point where
     * the ancestor would enter the same subtree as the target.
     */
    private Map<String, Map<String, Integer>> buildWeightedPackageGraph(
            DomainModel model, DependencyModel rawModel) {
        Map<String, Map<String, Integer>> weights = new HashMap<>();
        for (String pkg : model.getAllPackages().keySet()) weights.put(pkg, new LinkedHashMap<>());

        Set<String> allPkgNames = weights.keySet();

        for (DomainModel.CalculatedElementInfo cls : model.getAllClasses().values()) {
            String leafPkg = extractPackageName(cls.fullName);
            if (leafPkg == null || !allPkgNames.contains(leafPkg)) continue;

            // Count method calls per target package using raw model data
            Map<String, Integer> callCountPerPkg = new LinkedHashMap<>();
            DependencyModel.ClassInfo rawCls = rawModel.getClass(cls.fullName);
            if (rawCls != null) {
                for (DependencyModel.MethodInfo method : rawCls.methods.values()) {
                    for (Map.Entry<String, Integer> call : method.methodCalls.entrySet()) {
                        // Method call key format: "ownerClass.methodName"
                        // Find the target class by checking known dependencies
                        String calledKey = call.getKey();
                        for (String dep : cls.dependencies) {
                            if (calledKey.startsWith(dep + ".")) {
                                String toPkg = extractPackageName(dep);
                                if (toPkg != null && !leafPkg.equals(toPkg)
                                        && allPkgNames.contains(toPkg)) {
```

```
                                callCountPerPkg.merge(toPkg, call.getValue(), Integer::sum);
                            }
                            break;
                        }
                    }
                }
            }
        }
        // Fall back to 1 per target package for structural deps with no call data
        for (String dep : cls.dependencies) {
            String toPkg = extractPackageName(dep);
            if (toPkg != null && !leafPkg.equals(toPkg) && allPkgNames.contains(toPkg)) {
                callCountPerPkg.putIfAbsent(toPkg, 1);
            }
        }

        // Propagate each weight up to ancestor packages
        for (Map.Entry<String, Integer> entry : callCountPerPkg.entrySet()) {
            String toPkg = entry.getKey();
            int callCount = entry.getValue();
            String ancestor = leafPkg;
            while (ancestor != null && allPkgNames.contains(ancestor)) {
                if (ancestor.equals(toPkg) || toPkg.startsWith(ancestor + ".")) break;
                weights.get(ancestor).merge(toPkg, callCount, Integer::sum);
                ancestor = extractPackageName(ancestor);
            }
        }
    }
    return weights;
}

// ------------------------------------------------------------------------
// Step 5 -- reverse dependencies
// ------------------------------------------------------------------------

private void updateDependentRelationships(DomainModel model) {
    for (DomainModel.CalculatedElementInfo cls : model.getAllClasses().values()) {
        for (String dep : cls.dependencies) {
            DomainModel.CalculatedElementInfo d = model.getClass(dep);
            if (d != null) d.addDependent(cls.fullName);
        }
    }
    for (DomainModel.CalculatedElementInfo pkg : model.getAllPackages().values()) {
        for (String dep : pkg.dependencies) {
            DomainModel.CalculatedElementInfo d = model.getPackage(dep);
            if (d != null) d.addDependent(pkg.fullName);
        }
    }
}

// ------------------------------------------------------------------------
// Utilities
// ------------------------------------------------------------------------

private static boolean isInSameSubtree(String a, String b) {
    return a.startsWith(b + ".") || b.startsWith(a + ".");
}

private static String extractPackageName(String className) {
    if (className == null || !className.contains(".")) return null;
    return className.substring(0, className.lastIndexOf('.'));
}
```

```
370 }
```

Open Points Before Submission

- Confirm Veronika Schwarz as co-author and verify affiliation (e-mail: veronika.schwarz@qaware.de).
- Keep the submission abstract to one page in the INSTICC template.